

AI/ML & Backend Intern Assignment Report

Candidate Name: Sujitha

Date: May 08, 2026

1. Project Overview

This project is an AI-powered role-based candidate screening system. The application allows a candidate to upload a resume, select a target role, and attend a dynamic technical interview. The questions are generated using the candidate resume, selected role, and a Retrieval-Augmented Generation knowledge base built from the Section 9 books provided in the assignment.

2. Objective Implemented

The objective was to build a complete intelligent system integrating AI/ML concepts, backend API architecture, frontend interaction, data handling, and system flow management. The implemented system is modular and demonstrates an end-to-end interview lifecycle from resume upload to final PDF report generation.

3. System Flow

The candidate uploads a PDF or text resume. The backend parses the resume and extracts skills, technologies, domains, and a seniority signal. The candidate selects a target role such as AI/ML Engineer. The backend constructs a role-aware query, retrieves relevant knowledge chunks from the book-based knowledge base, generates interview questions, accepts candidate answers, stores the interaction, and provides final feedback and summary.

4. Backend Implementation

The backend is implemented using FastAPI. It exposes APIs for health check, role listing, knowledge ingestion, session creation, answer submission, session summary, and PDF report generation. The backend manages all business logic including resume parsing, profile extraction, retrieval, question generation, feedback generation, and persistence.

5. Frontend Implementation

The frontend is implemented using React. It provides a candidate-friendly interface for resume upload, role selection, interview question display, answer submission, profile signal display, source traceability, final summary, and PDF report download.

6. Data Layer and Storage

SQLite is used as the persistence layer. The database stores interview sessions, generated questions, candidate answers, retrieved context, feedback, profile information, and knowledge base chunks. The main tables are sessions, interactions, and knowledge_chunks. Questions and answers are saved automatically when the candidate submits an answer.

7. RAG Pipeline

The Retrieval-Augmented Generation pipeline uses the Section 9 books as the primary knowledge

source. The ingestion process extracts text from PDF books, splits the text into meaningful chunks, creates local embeddings, and stores the chunks and embeddings in SQLite. During the interview, the retriever searches across the ingested books and returns relevant chunks from multiple sources. The generated questions are grounded in these retrieved chunks.

8. Section 9 Knowledge Base Usage

The system uses the provided machine learning books as the RAG knowledge source, including Machine Learning by Tom Mitchell, The Hundred-Page Machine Learning Book, Introduction to Machine Learning with Python, Master Machine Learning Algorithms, Pattern Recognition and Machine Learning, and Artificial Intelligence, Machine Learning, and Deep Learning. The user interface shows source PDF names under each question for traceability.

9. Question and Answer Storage

Each generated question and submitted answer is stored in the SQLite database in the interactions table. The stored record includes session ID, question, answer, retrieved context, feedback, and timestamps. There is no manual save button because storage happens automatically when the Submit Answer button is clicked.

10. Output and Reporting

After the interview is completed, the system displays a structured summary and provides a Download PDF Report button. The report contains candidate profile signals, questions asked, answers given, feedback, and sources used. This makes it easy to verify stored interview records without manually opening the database.

11. Key Design Decisions

FastAPI was selected for the backend because it is lightweight, structured, and suitable for API-driven systems. React was selected for the frontend to provide a smooth interactive interview flow. SQLite was selected because it is simple, portable, and sufficient for assignment-level persistence. Local deterministic embeddings were used so the project can run without requiring paid external API keys.

12. Requirements Satisfaction

- Resume upload: Implemented.
- Role selection: Implemented.
- Resume parsing and profile extraction: Implemented.
- Role-specific RAG retrieval: Implemented.
- Knowledge ingestion and chunking: Implemented.
- Embedding generation and vector-like retrieval: Implemented.
- Dynamic question generation: Implemented.
- Interactive interview flow: Implemented.
- Session continuity: Implemented.
- Question and answer storage: Implemented.

- Structured summary: Implemented.
- Backend API design: Implemented.
- React frontend: Implemented.
- Database persistence: Implemented.
- Source traceability: Implemented.
- PDF report generation: Implemented.

13. Conclusion

The completed system satisfies the assignment requirements by combining a complete backend service, an interactive frontend, database persistence, a RAG pipeline using the provided books, dynamic interview question generation, answer storage, source traceability, and downloadable PDF reporting. The project demonstrates practical AI/ML integration with backend engineering and user-facing application design.